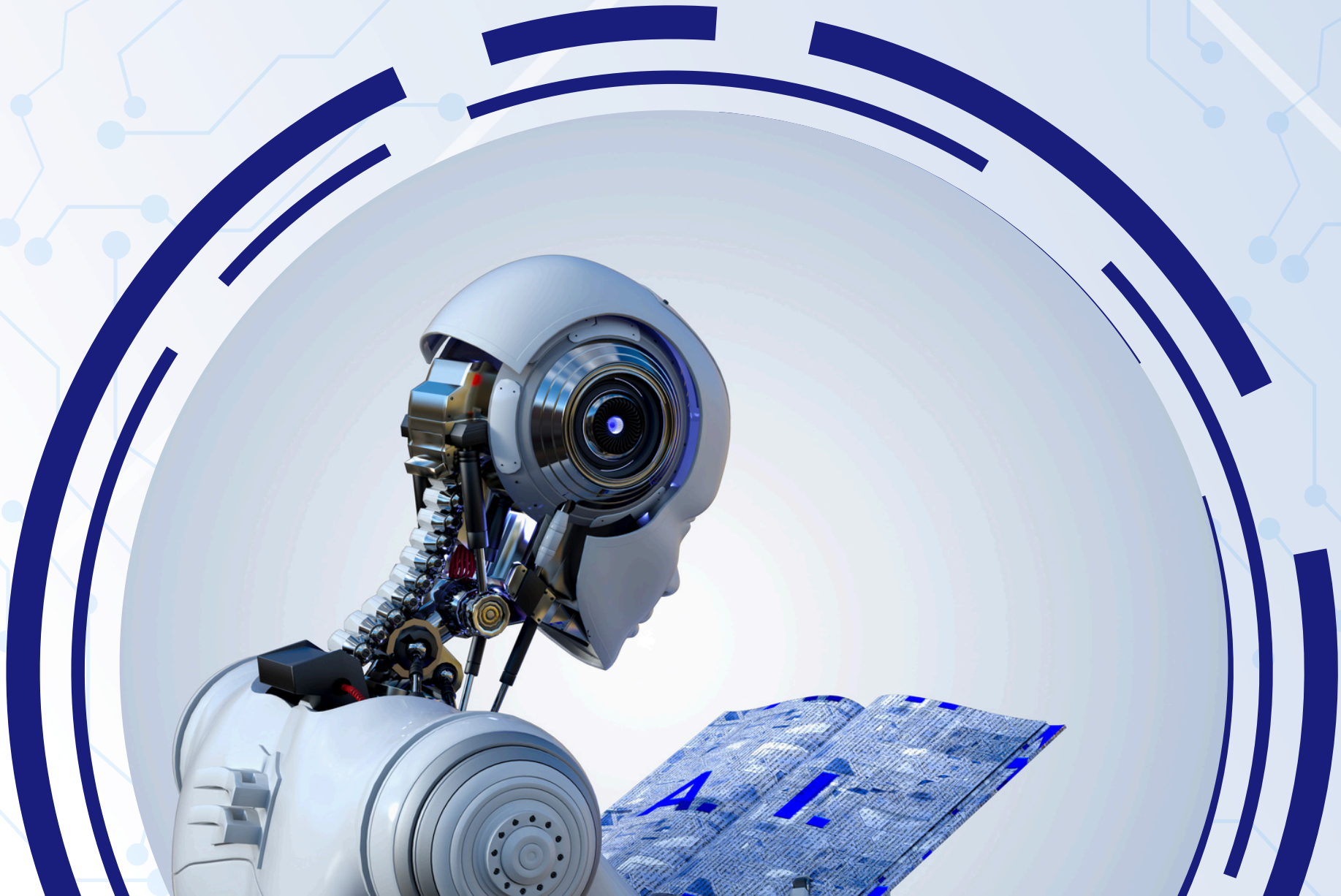
LoRA & QLoRA

What's the difference?

Let's break it down with math and real-world examples!



LoRA
(Low-Rank Adaptation)

What:

- A technique to fine-tune large language models (LLMs) efficiently by injecting low-rank matrices into the model's weights.

How it works:

- Decomposes weight updates (ΔW) into two smaller matrices: A and B.
- ΔW = A × B, where A and B are low-rank matrices.
- Instead of updating the full weight matrix (W), only A and B are trained, reducing the number of parameters.

Mathematical Intuition:

- If W ∈ ℝ^(m×n), then A ∈ ℝ^(m×r) and B ∈ ℝ^(r×n), where r << min(m, n).
- This reduces the number of trainable parameters from m×n to r×(m+n).

Use Cases:

- Fine-tuning LLMs for specific tasks (e.g., sentiment analysis, chatbots).
- Efficiently adapting models to new domains (e.g., medical, legal).

Real-World Example:

- Fine-tuning GPT-3 for customer support chatbots using LoRA reduces training costs while maintaining performance.

Example:

Suppose we have a GPT-3 model with a weight matrix W in one of its layers. Let's say W is a 4x4 matrix (for simplicity) that transforms input embeddings of size 4 into output embeddings of size 4. We want to fine-tune this model for a specific task, like customer support chatbot responses.

Weight Matrix (W):

$$W = \begin{bmatrix} 1.0 & 2.0 & 3.0 & 4.0 \\ 5.0 & 6.0 & 7.0 & 8.0 \\ 9.0 & 10.0 & 11.0 & 12.0 \\ 13.0 & 14.0 & 15.0 & 16.0 \end{bmatrix}$$

Low-Rank Decomposition:

Instead of updating the entire W (which would require 16 parameters), LoRA decomposes the weight update ΔW into two smaller matrices A and B with rank r = 2. This reduces the number of parameters to 8 (4x2 + 2x4).

Let's initialize A and B randomly:

$$A = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \\ 0.5 & 0.6 \\ 0.7 & 0.8 \end{bmatrix}, \quad B = \begin{bmatrix} 0.9 & 1.0 & 1.1 & 1.2 \\ 1.3 & 1.4 & 1.5 & 1.6 \end{bmatrix}$$

Weight Update (ΔW):

The weight update is computed as:

$$\Delta W = A \times B$$

Calculating ΔW:

$$\Delta W = \begin{bmatrix} 0.1 \times 0.9 + 0.2 \times 1.3 & 0.1 \times 1.0 + 0.2 \times 1.4 & \dots \\ 0.3 \times 0.9 + 0.4 \times 1.3 & \dots & \\ \vdots & \ddots & \end{bmatrix}$$

After performing the matrix multiplication, we get

$$\Delta W = \begin{bmatrix} 0.35 & 0.38 & 0.41 & 0.44 \\ 0.85 & 0.94 & 1.03 & 1.12 \\ 1.35 & 1.50 & 1.65 & 1.80 \\ 1.85 & 2.06 & 2.27 & 2.48 \end{bmatrix}$$

Updated Weight Matrix (W_new):

$$W_{\text{new}} = W + \Delta W = \begin{bmatrix} 1.0 + 0.35 & 2.0 + 0.38 & 3.0 + 0.41 & 4.0 + 0.44 \\ 5.0 + 0.85 & 6.0 + 0.94 & 7.0 + 1.03 & 8.0 + 1.12 \\ 9.0 + 1.35 & 10.0 + 1.50 & 11.0 + 1.65 & 12.0 + 1.80 \\ 13.0 + 1.85 & 14.0 + 2.06 & 15.0 + 2.27 & 16.0 + 2.48 \end{bmatrix}$$

The new weight matrix is:

$$W_{\text{new}} = \begin{bmatrix} 1.35 & 2.38 & 3.41 & 4.44 \\ 5.85 & 6.94 & 8.03 & 9.12 \\ 10.35 & 11.50 & 12.65 & 13.80 \\ 14.85 & 16.06 & 17.27 & 18.48 \end{bmatrix}$$

QLoRA
(Quantized LoRA)

What:

- An extension of LoRA that adds quantization to further reduce memory and computational costs.

How it works:

- Quantizes the model weights to lower precision (e.g., 4-bit or 8-bit) during fine-tuning.
- Combines quantization with LoRA's low-rank adaptation for even greater efficiency.

Mathematical Intuition:

- Quantization maps floating-point weights (e.g., 32-bit) to integers (e.g., 4-bit).
- For example, a weight value $w \in \mathbb{R}$ is mapped to $q \in \{0, 1, \ldots, 15\}$ for 4-bit quantization.
- Combined with LoRA: $\Delta W = \text{Quantize}(A \times B)$.

Use Cases:

- Fine-tuning LLMs on resource-constrained devices (e.g., edge devices, mobile phones).
- Reducing memory and compute requirements for large-scale deployments.

Real-World Example:

- Fine-tuning a medical LLM on a hospital's local server using QLoRA to analyze patient data without needing expensive GPUs.

Example:

Now, let's say we want to fine-tune the same GPT-3 model, but this time on a resource-constrained device (e.g., a smartphone). To save memory and compute, we use QLoRA, which quantizes the weights and updates to 4-bit precision.

Quantized Weight Matrix (W_q):

First, we quantize the original weight matrix W to 4-bit precision. Assume the quantization function maps values to integers between 0 and 15:

$$W_q = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

Low-Rank Decomposition:

We use the same A and B as before:

$$A = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \\ 0.5 & 0.6 \\ 0.7 & 0.8 \end{bmatrix}, \quad B = \begin{bmatrix} 0.9 & 1.0 & 1.1 & 1.2 \\ 1.3 & 1.4 & 1.5 & 1.6 \end{bmatrix}$$

Weight Update (ΔW):

The weight update is computed as:

$$\Delta W = A \times B$$

Calculating ΔW:

$$\Delta W = A \times B = \begin{bmatrix} 0.35 & 0.38 & 0.41 & 0.44 \\ 0.85 & 0.94 & 1.03 & 1.12 \\ 1.35 & 1.50 & 1.65 & 1.80 \\ 1.85 & 2.06 & 2.27 & 2.48 \end{bmatrix}$$

Quantized Weight Update (ΔW_q):

Now, we quantize ΔW to 4-bit precision. Using the same quantization function

$$\Delta W_q = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \end{bmatrix}$$

Updated Weight Matrix (W_new):

$$W_{\text{new}} = W_q + \Delta W_q = \begin{bmatrix} 1+0 & 2+0 & 3+0 & 4+0 \\ 5+1 & 6+1 & 7+1 & 8+1 \\ 9+1 & 10+2 & 11+2 & 12+2 \\ 13+2 & 14+2 & 15+2 & 16+2 \end{bmatrix}$$

Key Differences:

The new weight matrix is:

$$W_{\text{new}} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 6 & 7 & 8 & 9 \\ 10 & 12 & 13 & 14 \\ 15 & 16 & 17 & 18 \end{bmatrix}$$

Key Differences

Aspect	LoRA	QLoRA
Parameters	Low-rank matrices (A, B)	Low-rank matrices + Quantization
Memory Usage	Moderate reduction	Significant reduction
Precision	Full precision (e.g., 32-bit)	Low precision (e.g., 4-bit)
Use Cases	General fine-tuning	Resource-constrained fine-tuning

STATFUSION Ai

FOLLOW US ON

/ STATFUSIONAI